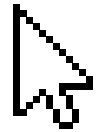
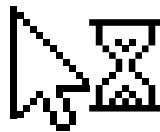




A* Pathfinding



Ganapathi Pamula | ADV Machine Learning and AI | Mr. Cochran





From BFS to A*

BFS

counts steps,
ignores cost

$$f = g + h$$

cost so far plus the
guess, lowest first



Guarantee

cheapest path if the
guess is never too
high

Dijkstra

adds real costs,
finds cheapest



A*

Dijkstra plus a
guess, h

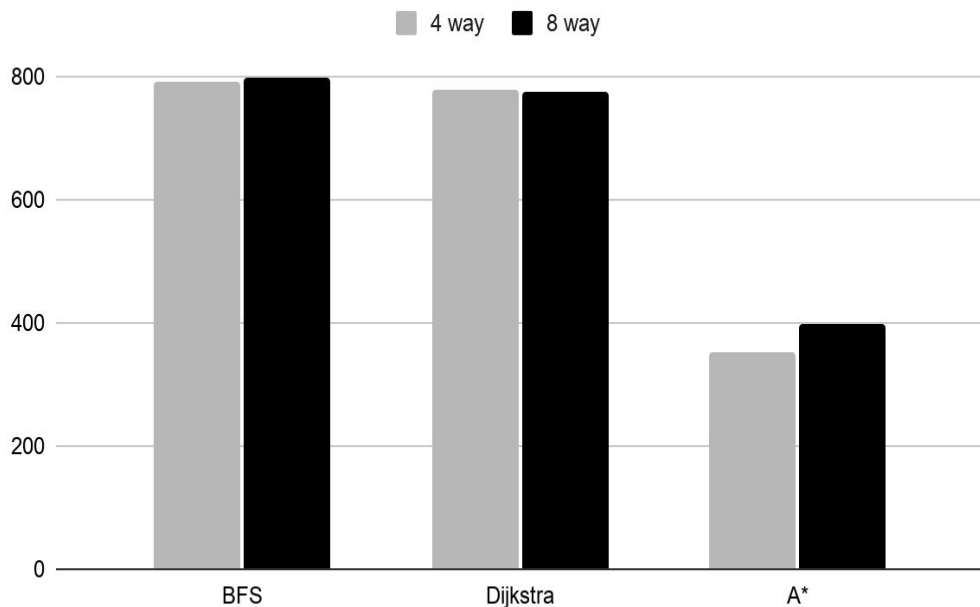
Limits

only as good as its
guess, uses lots of
memory



Same maps, three searches

Cells explored, average of 300 maps



BFS

fewest moves,
often more cost

Dijkstra

cheapest cost,
more searching

A*

same cost, about
half the cells



Is the guess ever too high?

Zero

always 0, same as
Dijkstra

Euclidean

straight line, never
too high

Chebyshev

longest side, never
too high

Manhattan

across + down, too
high with diagonals

Octile

counts diagonals,
never too high

Tiebreak

octile plus a tiny
nudge, can go a hair
high

Why fit beats speed

Manhattan

4 way: 351 cells,
always cheapest in
tests

Manhattan

8 way: 312 cells, not
cheapest on 28%

Weighted

4 way, guess x1.5:
191 cells, up to 14%
extra cost

Octile

8 way: 397 cells,
always cheapest in
tests

Chebyshev

4 way: 517 cells,
safe but less
focused

Greedy

4 way: 76 cells,
paths up to 2x the
cost

My adaptation experiment

66

Plain octile

8 way, (2,5) to
(37,28)

36

My adaptation

same map, cost
44.53

45%

Less work

cross product
idea: Red Blob
Games



Choices in the code

One loop

same loop,
different way to
pick



Stop at goal

stop only when
goal is picked

Skip old copies

outdated entries
get ignored

Pausable

animation and
tests share one
search

Tie order

nearer the goal
goes first

Edge cases

walled in goals,
no corner cutting

One loop, four algorithms

```
def priority(cell):  
    if algorithm == "BFS":  
        return steps[cell]  
    if algorithm == "Dijkstra":  
        return g[cell]  
    if algorithm == "Greedy":  
        return h(cell)  
    return g[cell] + weight * h(cell)  # A*
```

```
while frontier:  
    _, _, _ , current = heapq.heappop(frontier)  
    if current in closed:  # skip old copy  
        continue  
    closed.add(current)  
    result.expanded += 1  
    if current == goal:  # stop here  
        result.path = reconstruct_path(came_from, goal)  
        result.cost = g[goal]  
        break
```

How each one picks the next cell

BFS: fewest steps so far.

Dijkstra: cheapest cost so far (g).

Greedy: best guess to the goal (h).

A*: cost so far + guess (g + h).

Skip old copies

If I find a better route to a cell, I add it again. The old, worse copy just gets skipped.

Stop at the goal

The first path that reaches the goal might go through mud, so I only stop once the goal is picked next.



Mud, diagonals and my tiebreak

```
grid.py

def cost(self, cell):
    return MUD_COST if cell in self.mud else OPEN_COST

for dx, dy in DIAGONAL:
    nxt = (x + dx, y + dy)
    side_a = (x + dx, y)
    side_b = (x, y + dy)
    if (self.passable(nxt)
        and self.passable(side_a)
        and self.passable(side_b)):
        yield nxt, self.cost(nxt) * SQRT2
```

Mud and diagonals

Mud costs 5 instead of 1, so the shortest path is not always the cheapest. A diagonal step costs about 1.41, and the side checks stop a path from cutting between two walls.

```
heuristics.py

def tiebreak_octile(node, goal, start):
    h = octile(node, goal)
    dx1, dy1 = node[0] - goal[0], node[1] - goal[1]
    dx2, dy2 = start[0] - goal[0], start[1] - goal[1]
    cross = abs(dx1 * dy2 - dx2 * dy1)
    return h + cross * 0.001
```

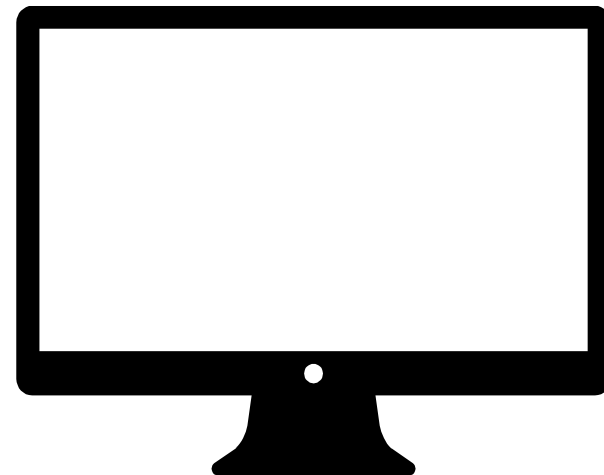
My tiebreak

When two cells look equally good, it picks the one closer to the straight line from start to goal. On an open map: 66 cells searched down to 36, same cost.



The visualizer

Blue shows exploration order.
Yellow shows the path. Red warns
about Manhattan with diagonals.
TAB compares work and cost





The repo

Platform Solutions Resources Open Source Enterprise Pricing

gspamula / Project1_Pathfinding Public

Code Issues Pull requests Actions Projects Security and quality Insights

main 1 Branch 0 Tags

Go to file

Code

gspamula Add the Cowork chat where the project was first built to the AI logs 8b818eb · 5 days ago 8 Commits

pathfinding Document benchmark and tests, update AI-use notes in h... 5 days ago

results Rerun the benchmark 5 days ago

tests Document benchmark and tests, update AI-use notes in h... 5 days ago

.gitignore Initial commit: A* pathfinding project last week

2026-09-14_cowork_project_build.md Add the Cowork chat where the project was first built to th... 5 days ago

2026-09-16_pycharm_and_git_setup.md Add AI chat logs for Honor Code documentation 5 days ago

2026-09-17_python_interpreter_error.md Add AI chat logs for Honor Code documentation 5 days ago

2026-09-17_rubric_check_comments_and_u... Add AI chat logs for Honor Code documentation 5 days ago

AI_CHAT_LOGS.md Add the Cowork chat where the project was first built to th... 5 days ago

README.md README: Implementation decisions, corrected claims, AI u... 5 days ago

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Report repository

Releases

No releases published

Contributors 2

gspamula Ganapathi

claude Claude

window.py Redesign the visualizer and make it sharp on Retina screens 5 days ago

README

Project 1: A* Pathfinding Exploration

ADV Machine Learning and AI, Mr. Cochran

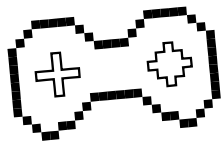
An A* pathfinding implementation written from scratch in Python, with a live PyGame visualizer and a benchmark that measures how different heuristics trade speed for accuracy. BFS, Dijkstra, and Greedy Best First Search are included for comparison.



Algorithm	Heuristic	Weight	Movement	Speed
A*	Octile	1x (default)	8 directions	10 cells per frame

Algorithm	Score	Time
BFS	714	80.0
Dijkstra	714	80.0
Greedy Octile	440	80.0
A* Manhattan	440	80.0
A* Euclidean	440	80.0
A* Octile	440	80.0

My README explains choices, results and run steps. All 128 tests pass. Code comments and chat logs document the work and AI help



What I learned

1. I match the heuristic to the movement rules
2. I check path cost before calling a method better
3. I learned to debug the setup as well as the code



Claude assisted with formatting and functioned as a brainstorming partner for these slides.

CREDITS: This presentation template was created by
Slidesgo, and includes icons by **Flaticon**, and infographics
& images by **Freepik**



CREDITS: This presentation template was created by
Slidesgo, and includes icons by **Flaticon**, and infographics
& images by **Freepik**